INVENTORY MANAGEMENT SYSTEM

Case Study Report – REST API Based Inventory Web Application

Submitted for: College Practical / Case Study Evaluation

Marks Distribution: Architecture 20 | Webpage 20 | Postman 20 | Report 20 | Demo 20

Student Name: _____Deepak R_____
Register No.: _____192511008_____
Department: Computer Science and Engineering

1. Abstract

The Inventory Management System is a lightweight web application developed to manage products, stock quantities, prices, categories, and suppliers through a simple dashboard. The system uses HTML, CSS, and vanilla JavaScript for the frontend and a Node.js with Express.js REST API for the backend. Inventory information is stored in a JSON file, making the implementation easy to deploy and understand for an academic practical. The application supports CRUD operations—Create, Read, Update, and Delete—and the REST API is tested using Postman. The system also calculates total products, total stock, low-stock items, and total inventory value, while providing search, category filtering, and stock-status indicators.

2. Problem Statement

Manual inventory tracking can make it difficult to maintain accurate product quantities, prices, supplier information, and stock status. A simple computerized inventory application is required to provide centralized product information, basic stock monitoring, and REST-based operations that can be tested independently using an API client.

3. Objectives

- Develop a responsive inventory management webpage using HTML, CSS, and JavaScript.
- Implement RESTful CRUD operations using Node.js and Express.js.
- Store inventory data in a simple JSON file for local persistence.
- Connect the frontend to the backend using the Fetch API and JSON.
- Provide search, category filtering, stock status, and dashboard statistics.
- Test all major REST API operations using Postman.
- Prepare a clear architecture, workflow, report, and demonstration.

4. Technologies Used

Technology	Purpose
HTML5	Structure of the inventory webpage
CSS3	Responsive layout and visual styling
JavaScript	Frontend logic, CRUD actions, search and filtering
Node.js	Backend JavaScript runtime
Express.js	REST API framework
JSON	Local inventory data storage
Fetch API	Frontend-to-backend HTTP communication
Postman	REST API testing

5. System Architecture

The system follows a simple client-server architecture with a frontend layer, REST API layer, and local data layer. Postman connects directly to the REST API for independent testing.

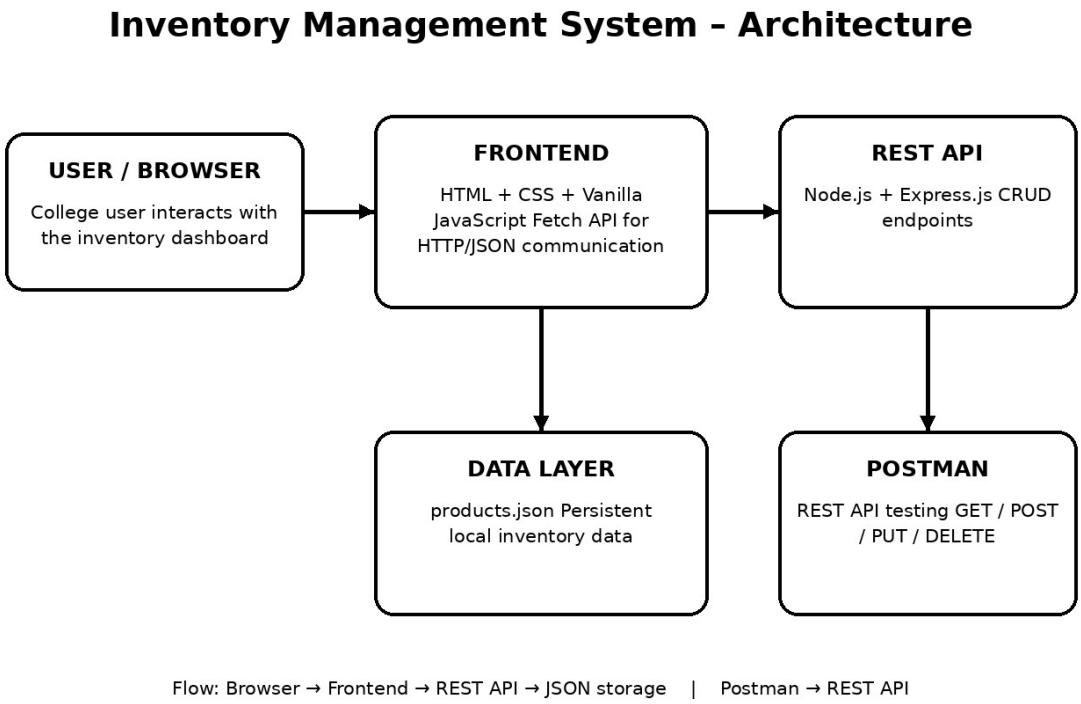

Figure 1. Architecture of the Inventory Management System

6. Components

Component	Description
User / Browser	Interacts with the inventory dashboard.
Frontend	HTML, CSS and JavaScript dashboard used to display and modify inventory.
REST API	Node.js/Express.js server exposing CRUD endpoints.
Data Layer	products.json file containing product records.
Postman	Tool used to send HTTP requests and verify API responses.

7. System Workflow

View Products

The browser loads the webpage and JavaScript sends a GET request to /api/products. The API reads products.json and returns JSON data, which is rendered in the table.

View One Product

The client sends GET /api/products/:id. The API searches for the requested ID and returns the matching product or a 404 response.

Add Product

The user enters product information. JavaScript sends a POST request with a JSON body. The server validates the fields, assigns an ID, saves the product, and returns the created record.

Update Product

The client sends a PUT request containing the updated fields. The API locates the ID, updates the record, saves the JSON file, and returns the updated product.

Delete Product

The client sends DELETE /api/products/:id. The API removes the record and returns a success response. A later GET for the same ID returns 404.

Search and Filter

The frontend filters the displayed inventory by product name/category and updates the visible table without changing the stored records.

8. Inventory Webpage

The dashboard provides a practical interface for inventory operations. The observed implementation contains summary cards, search, category filtering, an Add Product control, an inventory table, stock-status indicators, and edit/delete actions.

Dashboard values verified from the initial dataset:

Metric	Value
Total Products	9
Total Stock	105 units
Low Stock Products	2
Out of Stock	1
Total Inventory Value	₹13,90,900

Stock-status rule used in the implementation: quantity 0 = Out of Stock; quantity 1–5 = Low Stock; quantity greater than 5 = In Stock.

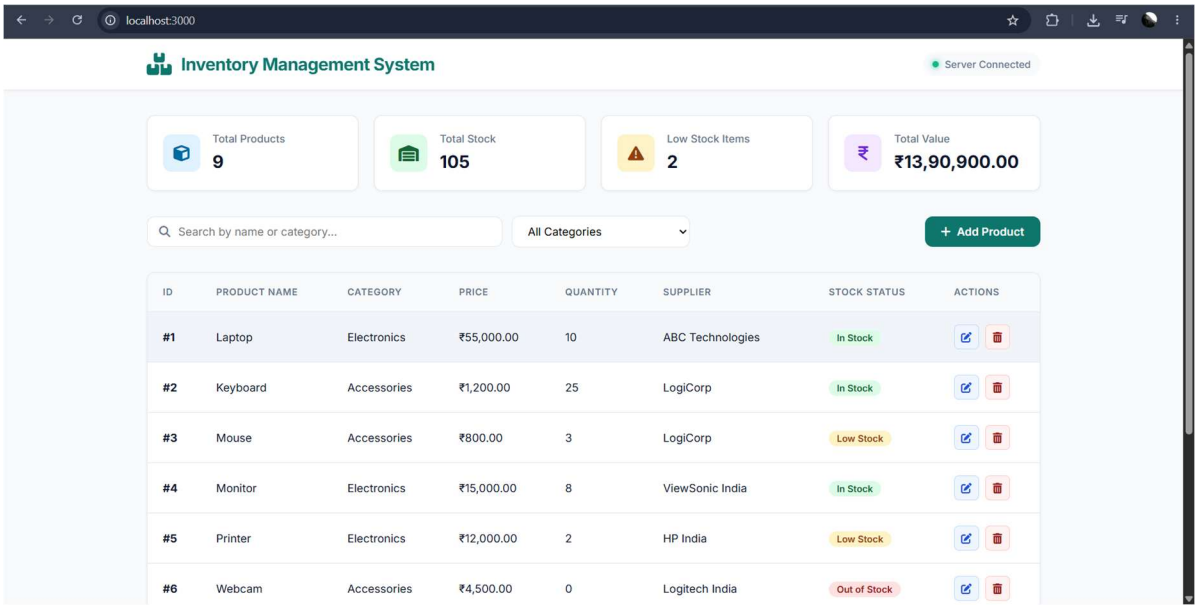

Figure: Inventory Management System interface (captured during implementation).

Figure: Inventory Management System interface (captured during implementation).

9. REST API Design

Method	Endpoint	Purpose	Expected Success
GET	/api/products	Get all products	200 OK
GET	/api/products/:id	Get one product	200 OK
POST	/api/products	Create a product	201 Created
PUT	/api/products/:id	Update a product	200 OK
DELETE	/api/products/:id	Delete a product	200 OK or 204 No Content
GET	/api/products/search/:keyword	Search products	200 OK
GET	/api/categories	Get categories	200 OK

10. Product Data Structure

Each inventory record contains the following fields:

Field	Type	Description
id	Integer	Unique product identifier
name	String	Product name
category	String	Product category
price	Number	Unit price
quantity	Integer	Available stock quantity
supplier	String	Supplier name

11. API Testing Using Postman

The following tests were executed against the local REST API at http://localhost:3000.

Test	Request	Observed Result
Get all products	GET /api/products	Returned all 9 inventory records

		as JSON.
Get product by ID	GET /api/products/1	Returned Laptop record with ID 1.
Create product	POST /api/products	Created USB Hub with ID 10.
Update product	PUT /api/products/10	Updated USB Hub to USB Hub Pro, price ₹2,000, quantity 25.
Delete product	DELETE /api/products/10	Temporary test product was deleted successfully.
Verify deletion	GET /api/products/10	Returned message: Product with ID 10 not found.

12. Postman Test Evidence

For the final printed/submitted report, insert the actual Postman screenshots under the corresponding captions below. The API responses were successfully verified during testing.

Figure 2. GET /api/products – all products returned as JSON.

[INSERT POSTMAN SCREENSHOT HERE]

Figure 3. GET /api/products/1 – single product returned.

[INSERT POSTMAN SCREENSHOT HERE]

Figure 4. POST /api/products – USB Hub created with ID 10.

[INSERT POSTMAN SCREENSHOT HERE]

Figure 5. PUT /api/products/10 – USB Hub updated to USB Hub Pro.

[INSERT POSTMAN SCREENSHOT HERE]

Figure 6. DELETE /api/products/10 – temporary product deleted.

[INSERT POSTMAN SCREENSHOT HERE]

Figure 7. GET /api/products/10 after deletion – 404 / product not found verification.

[INSERT POSTMAN SCREENSHOT HERE]

13. Validation and Error Handling

- Product name, category, and supplier should not be empty.
- Price should be zero or greater.
- Quantity should be a non-negative integer.
- Invalid or missing product IDs should return a suitable not-found response.
- Invalid routes should be handled by the server.
- Frontend operations should display user-friendly success or error feedback.

14. Security Analysis

- Input validation reduces the risk of invalid inventory records.
- Only expected JSON fields should be accepted by the API.

- HTTP methods are used according to their intended CRUD operations.
- Sensitive credentials are not required because this is an academic local prototype.
- For production deployment, HTTPS, authentication, authorization, rate limiting, database access controls, and stronger validation should be added.

15. Results

The implemented system successfully displayed inventory records through a web dashboard and communicated with the Express REST API. The API was verified for reading all products, reading an individual product, creating a new product, updating a product, and deleting a temporary test product. The final verification request confirmed that the deleted product was no longer available. The initial inventory contained 9 products, 105 total units, 2 low-stock products, 1 out-of-stock product, and an inventory value of ₹13,90,900.

16. Advantages

- Simple and easy-to-understand architecture.
- CRUD operations provide complete basic inventory management.
- Frontend and REST API are independently testable.
- Postman provides clear API verification.
- JSON storage keeps the academic prototype lightweight.
- Search, filtering, and stock indicators improve usability.

17. Limitations

- JSON file storage is not suitable for high-concurrency production systems.
- No user authentication or role-based access control is implemented.
- The system is designed for local/academic demonstration.
- No advanced reporting, barcode integration, or cloud synchronization is included.

18. Future Scope

- Replace JSON storage with MySQL, PostgreSQL, or MongoDB.
- Add authentication and admin/user roles.
- Add purchase and sales transactions.
- Add barcode/QR scanning.
- Add low-stock notifications.
- Add cloud deployment and centralized database backup.
- Add analytics dashboards and downloadable inventory reports.

19. Demo / Presentation Sequence

Time	Demonstration
0:00–0:30	Introduce the problem and project objective.
0:30–1:30	Explain architecture diagram and technologies.
1:30–3:00	Show dashboard, search, category filter, stock status, add/edit/delete.
3:00–4:30	Open Postman and demonstrate GET, GET by ID,

	POST, PUT and DELETE.
4:30–5:30	Verify deletion and show frontend/data consistency.
5:30–6:00	Explain results, limitations and future scope.

20. Viva Questions and Short Answers

Q: What is a REST API?

A: A REST API is an HTTP-based interface that allows clients and servers to exchange resources, commonly using JSON.

Q: What does CRUD mean?

A: Create, Read, Update, and Delete.

Q: What is GET used for?

A: GET retrieves data from the server.

Q: What is POST used for?

A: POST creates a new resource.

Q: What is PUT used for?

A: PUT updates an existing resource.

Q: What is DELETE used for?

A: DELETE removes a resource.

Q: Why is JSON used?

A: JSON is lightweight, human-readable, and easy for JavaScript and REST APIs to exchange.

Q: What is Express.js?

A: Express.js is a Node.js web framework used to create routes and REST APIs.

Q: What is Node.js?

A: Node.js is a JavaScript runtime used to execute JavaScript on the server.

Q: What is Fetch API?

A: Fetch API lets browser JavaScript send HTTP requests and process responses.

Q: What is Postman?

A: Postman is a tool for creating, sending, and validating API requests.

Q: Why use products.json?

A: It provides simple local persistence without requiring a database for this academic prototype.

Q: What does 404 mean?

A: The requested resource was not found.

Q: What does 201 mean?

A: A resource was successfully created.

Q: What is an endpoint?

A: An endpoint is a specific URL through which an API operation can be accessed.

Q: What is client-server architecture?

A: The client sends requests and the server processes them and returns responses.

Q: How does the frontend communicate with the backend?

A: JavaScript uses Fetch API to send HTTP requests to the Express REST API.

Q: How is low stock identified?

A: In this implementation, quantity 1–5 is treated as low stock and quantity 0 as out of stock.

Q: Why should production systems use a database?

A: Databases provide better concurrency, querying, indexing, integrity, security, and scalability.

Q: What happens after deleting product 10?

A: A subsequent GET for ID 10 returns a not-found response, confirming deletion.

21. Conclusion

The Inventory Management System successfully demonstrates a complete small-scale web application with a frontend, REST API, local data layer, and API testing workflow. The project satisfies the core requirements of the case study by providing a functional inventory webpage, a RESTful backend, CRUD operations, Postman testing, architecture documentation, and a clear demonstration flow. The implementation is intentionally lightweight so that each component can be understood and explained during a college practical examination.

22. Submission Checklist

☐ Architecture diagram included.

☐ Inventory webpage tested in browser.

☐ GET all products tested in Postman.

☐ GET product by ID tested in Postman.

☐ POST product tested in Postman.

☐ PUT product tested in Postman.

☐ DELETE product tested in Postman.

☐ Post-delete 404 verification captured.

☐ Frontend screenshots inserted.

☐ Postman screenshots inserted.

☐ README/project files included.

☐ Demo and viva preparation completed.